

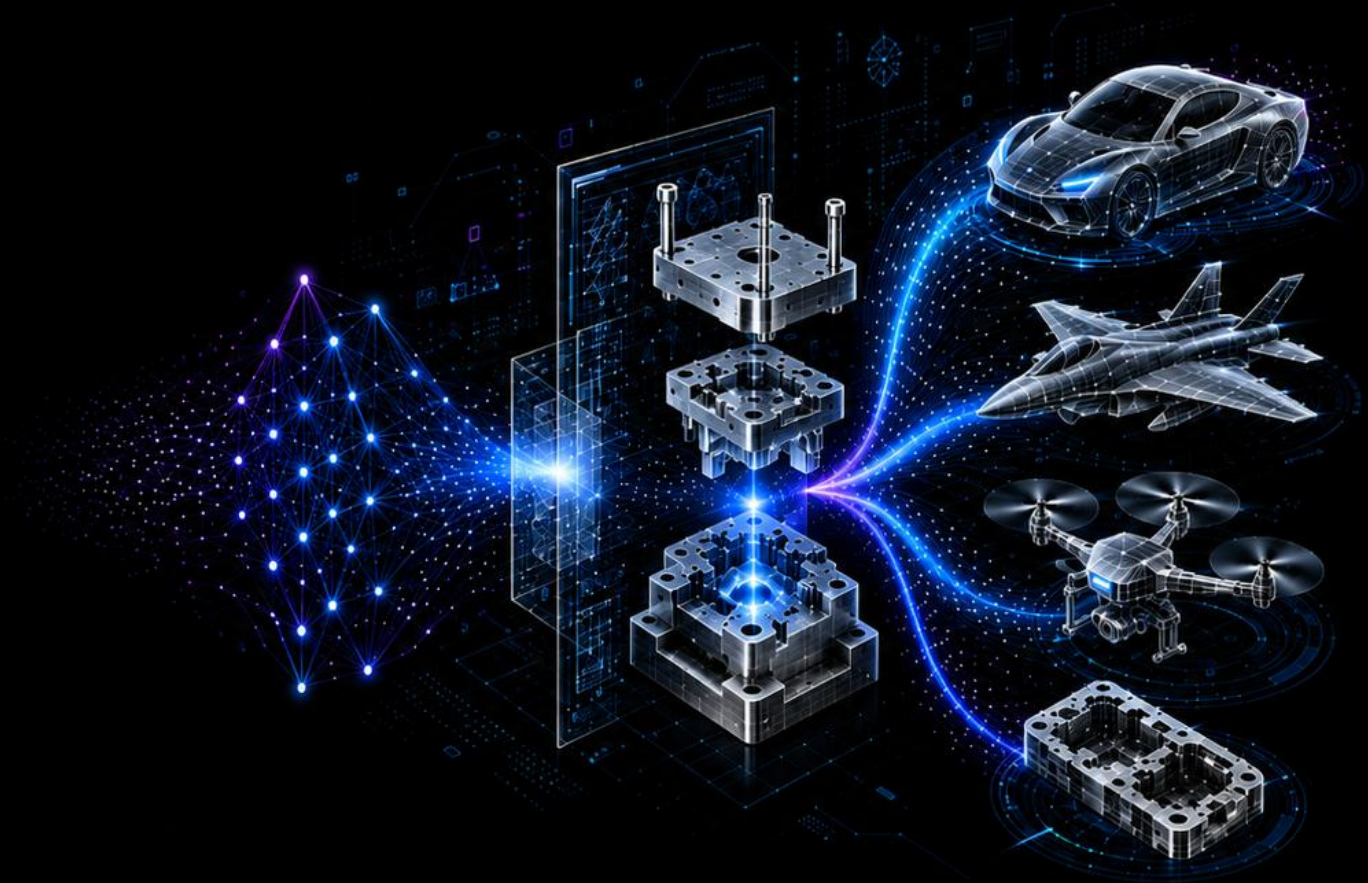
개인 과제 발표

Text-to-CAD

최준석

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과



Intelligent Design
for Engineering
Applications Lab

■ 진행 상황

이번주는 “형상 계획 단계 통과”에 집중

이유: 정답지를 제공했을 때 이상 없음을 확인했기 때문에, 이제는 어떤 부분 때문에 안되는 지 원인을 분석하기 위함

1. 순수 gpt-oss모델의 형상 계획 작성 능력 파악과 형상계획문의 기본 틀 제공의 필요성 (완료)
2. 자연어 요청 한글 vs 영어 에 따른 형상계획문 결과 비교 (완료)
3. GPT-oss:20b 모델로 변경했음에도 형상문을 통과하지 못하는 결정적인 원인 분석과 개선 연구 (진행 중)

■ 현재의 큰 문제는 형상 계획 작성 단계에서 문제 발생

- 단순히 gpt-oss를 통해 전체 파이프라인 실행 시, **형상 계획 단계에서부터 실패하는 문제 발생**
- 해결 방안: 코덱스가 작성한 정답지를 기준으로 형상문의 최소 틀만 제공해보자 라고 판단

코덱스가 작성한 정답지

```
{
  "schema_version": "2.0",
  "title": "Reference four-part airframe",
  "coordinate_system": "X_AFT_Y_RIGHT_Z_UP",
  "units": {
    "length": "m",
    "angle": "deg"
  },
  "components": [
    {
      "id": "fuselage",
      "source_component_id": "fuselage",
      "name": "Fuselage",
      "openvsp_geom_type": "FUSELAGE",
      "parent_component_id": null,
      "plan": "Create one FUSELAGE with its nose at the origin and its longitudinal axis pointing toward the tail along +X. Retain the default zero translation and rotation without adding symmetry copies. Set Length in Design to 10 m. Reuse the five default cross sections: point nose at fraction 0, elliptical sections at fractions 0.25, 0.5 and 0.75, and point tail at fraction 1 of the length; insert or delete no sections. On each of XSecCurve_1, XSecCurve_2 and XSecCurve_3, set Ellipse_Width to 1.5 m and Ellipse_Height to 1.5 m, giving a circular middle body with maximum width 1.5 m and maximum height 1.5 m. Retain the default section centers and nose/tail skinning controls to form closed tapered ends. Keep this fuselage and the main wing in the same model, with the wing root inside the middle body.",
      "source_ids": [
        "geometry_planning:geometry.json"
      ]
    },
    {
      "id": "main_wing",
      "source_component_id": "main_wing",
      "name": "Main wing",
      "openvsp_geom_type": "WING",
      "parent_component_id": null,
      "plan": "Create one WING in the same model as the fuselage and retain its default XZ planar symmetry across Y=0. Set X_Rel_Location to 4 m, Y_Rel_Location to 0 m and Z_Rel_Location to 0 m in XForm with zero rotation, placing the root leading edge 4 m aft of the fuselage nose on its centerline and at the same height. The root overlaps the fuselage middle body. Reuse the existing single spanwise section XSec_1; insert no sections. Select span, root chord and tip chord as the three independent section drivers. Set section Span to 5 m, Root_Chord to 2 m and Tip_Chord to 1 m, giving a full span of 10 m across both sides. Set Sweep to 15 deg at 25 percent of the chord, equivalent to Sweep_Location 0.25. Retain the four-series airfoil type and set Camber 0 and ThickChord 0.12 on both XSecCurve_0 and XSecCurve_1 for NACA 0012. Evaluate the final assembled geometry after applying all changes."
    }
  ]
}
```

제공해준 최소 형상문 틀

```
{
  "schema_version": "2.0",
  "title": "형상 제목",
  "coordinate_system": "",
  "units": { },

  "components": [
    {
      "id": "",
      "source_component_id": "",
      "name": "",
      "openvsp_geom_type": "",
      "parent_component_id": null,
      "plan": "",
      "source_ids": ["사용한_RAG_자료_ID"]
    }
  ],

  "calculations": [
    {
      "description": "계산 설명",
      "expression": "",
      "result": ,
      "unit": ""
    }
  ],

  "coverage": [
    {
      "source_component_id": "",
      "requirement_names": [ "", "", "", "", "" ],
      "relationship_texts": [ "", "" ],
      "global_constraints": [ ]
    }
  ],

  "unresolved": [ ]
}
```

■ 두 가지 실험의 비교결과 및 필요성 조사

- gpt-oss가 순수하게 작성한 형상문과 기본 틀만 제공했을 때, 코덱스가 작성한 정답지와 48개 항목·기준으로 비교

정답지와 일치율

조건	seed103	seed104	seed105	평균	일치율
A 틀 없음	25.5	26.0	25.0	25.5/48	53%
B 틀 제공	34.0	38.0	36.5	36.2/48	75%

항목군	만점	A	B
문서 8필드	8	4.7	8.0
component 7필드	7	1.0	5.2
동체 내용	4	2.8	3.0
주익 내용	7	5.0	5.8
수평꼬리 내용	7	5.0	5.8
수직꼬리 내용	8	5.8	6.3
calculations·coverage	6	1.2	2.0
JSON 파싱	1	0	0

전체 세부 결과

항목	A(틀 없음)	B(틀 제공)
문서 최상위 구조	✗ geometry_plan 래퍼, schema_version · calculation s · coverage 0/3	✓ 8필드 3/3 정답 위치
component 필드	✗ 해석문의 role / description / parameters 모방, openvsp_geom_type · plan · source_ids 없음	✓ 7필드 3/3
plan 서술	✗ 0/3 (수치 객체)	○ 2/3 자연어 문장 (103만 객체)
요청 수치(시위·스팬·후퇴각 값)	✓ 3/3 정확	✓ 3/3 정확
위치 (4,0,0)/(8,0,0)	✓ 3/3	✓ 3/3
편측 스펠 변환(총 10→5)	✗ 0/3 명시	○ 1/3 (104: "5 m on each side")
등록 Geom 명 FUSELAGE/WING	✗ 타입 자체 없음	✗ cylinder / fuselage / wing
Parm(Sweep_Location=0.25, Camber / ThickChord=0.12, XSecCurve)	✗ 0/3	✗ 0/3
단일 수직꼬리 (Sym_Planar_Flag=0 + X_Rel_Rotation=+90)	✗ (105는 YZ 대칭 지정)	✗ (103-104는 "XZ 대칭" 명시 → 핀 2장)
수직꼬리 +Z 방향 표현	○ 2/3 (단면 z 0→1.8)	○ 2/3 ("upward along +Z")
calculations	✗ 배열 자체 없음	✗ 3개 생성했으나 값 전부 오류(2.5/1.0, 0.5/0.3/0.6) 또는 계산 아님
coverage 연결	✗ 없음	○ requirement_names 3/3, ✗ relationship_texts 전부 빈 배열 · source_component_id 빈 문자열
JSON 파싱	✗ 3/3 코드블록, 105는 Math.tan() 식 삽입	✗ 3/3 코드블록
근거 없는 추가 설계	✗ "biplane"(104)	✗ 반구 노즈·중앙부 8 m(104), 요청에 없는 중간 단면(103)

■ 두 가지 실험의 비교결과 및 필요성 조사

- gpt-oss가 순수하게 작성한 형상문과 기본 틀만 제공했을 때, 코덱스가 작성한 정답지와 48개 항목·기준으로 비교

형상계획문 틀의 필요성

- 형상 계획문을 불필요하게 복잡하게 쓰려는 경향은 gpt-oss·qwen 모두에서 확인했고, 상위 모델일수록 간결하게 적는 경향을 보임.

코덱스가 작성한 비교 기준이 된 정답지는 OpenVSP에서 실행하고 문제 없음을 확인함.

- 틀을 제공하면 출력 문서 계약이 고정되어 정답지 일치율이 53% → 75%로 향상됐다.

반면 Geom 등록명칭이나 계산식 등은 양쪽 모두 실패해, 틀은 형식의 필요 조건이며, 형상 정확도는 틀 제공 이외의 문제임을 확인했다.

(해당 실패가 공식 지식 부족인지 지식 활용 실패인지는 RAG를 제공한 조건에서 별도 검증이 필요하다.)

	순수 작성(A)	정답지
최대 중첩 깊이	8단 (geometry_plan→components→parameters→cross_sections→...)	5단
주요 블록 키 개수	25개	7개
주요 표현 길이	599자	1,084자

위는 복잡성에 관한 비교 결과표, 왼쪽 아래는 순수 작성 / 오른쪽은 정답지

```
{
  "id": "main_wing",
  "role": "wing",
  "description": "Symmetric main wing about XZ plane",
  "parameters": {
    "span": 10.0,
    "root_chord": 2.0,
    "tip_chord": 1.0,
    "sweep_25pct": 15.0,
    "leading_edge_location": [4.0, 0.0, 0.0],
    "airfoil_root": "NACA 0012",
    "airfoil_tip": "NACA 0012",
    "cross_sections": [
      {
        "position_along_span": 0.0,
        "chord": 2.0,
        "sweep_offset": 0.0,
        "airfoil": "NACA 0012"
      },
      {
        "position_along_span": 5.0,
        "chord": 1.0,
        "sweep_offset": 0.0,
        "airfoil": "NACA 0012"
      }
    ]
  },
  "sweep_calculation": {
    "reference_chord": 0.25,
    "sweep_angle": 15.0,
    "sweep_offset_at_root": 0.0,
    "sweep_offset_at_tip": 0.0
  }
}
```

```
{
  "id": "main_wing",
  "source_component_id": "main_wing",
  "name": "Main wing",
  "openvsp_geom_type": "WING",
  "parent_component_id": null,
  "plan": "Create one WING in the same model as the f",
  "source_ids": ["geometry_planning:geometry.json"]
}
```

■ 형상문 틀만 제공했을 시의 비교 결과

- 둘다 완벽하게 작성은 하지 못함. 하지만 **미세하게 영어가 한글보다는 나은 점을 보임**
- 정답지 기준: 수직꼬리는 루트가 Z=0이고 Span 1.8m로 위로 뻗는다

한글

- ❖ 루트가 위치 자체가 위로 올라가라는 것처럼 받아들임.

n the centerline ($Y=0$) and the root extends upward 1.8 m (+Z). The root chord

영어

- ❖ Z=0이라는 명시는 못 적었지만, 루트에서 위로 1.8m 뻗는다 라고 잘 작성함

f the nose. The tail extends 1.8 m upward along +Z from the root. It has a root chord

⇒ 자연어 요청은 영어로 일단 고정하기로 결정

형상 계획문을 어떻게 못 적는가?

■ 기본 형상 틀(RAG) + 자연어 영어 요청만 제공했을 때, 비교 결과표

#	항목	작성문	정답지	판정
1	부품 구성("id")	:11~ 4부품	4부품	✓
2	전체적인 치수	:16:25:34:43 10/1.5-10/ 2/1/15-4/1.2/0.6-1.8/1.5 /0.7/25	동일	✓
3	Geom 식별자	:14 "fuselage" / "wing" "	:14 "FUSELAGE" / "WING" "	X
4	Parm 지정	없음	:16:27 Length · Ellipse_W/H · Span · Sweep_Location 0.25 · Camber 0 · ThickChord 0.12 · 드라이버·XSec_1	X
5	문장 성격	:16 "A straight fuselage with..." — 완성 형상 묘사	:16 "Create one FUSELAGE... Set Length..." — 구현 지시	X
6	자율 설계 추가	:16 "선형으로 가늘어지는 대칭 타원 단면", :25 "linear chord variation" 등 요청에 없는 서술	기본 5단면 유지 등 근거 있는 선택만	X
7	Z 위치·대칭 해제	주요·수평꼬리 Z=0 수치 없음, 수직꼬리는 "single"만 언급	:27 Z_Rel_Location 0, :52 Sym_Planar_Flag 0 + X_Rel_Rotation +90	X
8	source_ids	:17 [] 공란	:17 자료 ID	X
9	calculations	:47 [] 공란	:55 5 m-2 m	X
10	coverage	:48 [] 공란	:69 4부품 요구·관계	X

형상 계획문을 어떻게 못 적는가?

■ 기본 형상 틀(RAG) + 자연어 영어 요청만 제공했을 때, 비교 결과표

#	항목	작성문	정답지	판정
1	부품 구성("id")	:11~ 4부품	4부품	✓
2	전체적인 치수	:16:25:34:43 10/1.5-10/ 2/1/15-4/1.2/0.6-1.8/1.5 /0.7/25	동일	✓
3	Geom 식별자	:14 "fuselage" / "wing"	:14 "FUSELAGE" / "WING"	X
4	Parm 지정	없음	:16:27 Length · Ellips e_W/H · Span · Sweep_Lo cation 0.25 · Camber 0 · ThickChord 0.12 · 드 라이버·XSec_1	X

Geom Browser
 POD
FUSELAGE
 WING
 STACK
 BLANK
 ELLIPSOID
 BODYOFREVOLUTION
 HUMAN
 PROP
 GEAR
 HINGE
 CONFORMAL
 ROUTING
 AUXILIARY
 COBRA

Add
 Delete
 Clipboard
 Copy
 Paste
 Cut
 Selection
 Sel All
 Pick
 Show Only
 Show
 NoShow

LLM 작성문 문제 예시

```

{
  "id": "fuselage",
  "source_component_id": "fuselage",
  "name": "Fuselage",
  "openvsp_geom_type": "fuselage",
  "parent_component_id": null,
  "plan": "A straight fuselage with its nose at the origin (0,0,0) and its tail extending",
  "source_ids": []
}
  
```

3번 예시 사진

올바른 정답지 예시

```

"id": "fuselage",
"source_component_id": "fuselage",
"name": "Fuselage",
"openvsp_geom_type": "FUSELAGE",
"parent_component_id": null,
"plan": "Create one FUSELAGE with its nose at the origin and its longitudinal ax",
"source_ids": ["geometry_planning:geometry.json"]
  
```

■ 현재 어떤 이유 때문에 형상 계획문 작성을 못하는가?

원인 분석 과정

- ❖ 자세한 원인 분석과 개선을 위하여 실험 조건을 총 3가지 조건으로 원인 분석을 진행하였다.
- ❖ 실험과정은 아래 총 3가지 조건으로 진행, 형상계획 틀만 제공했을 때, 정답지와 비교하여 틀린 부분을 3가지 조건으로 해결해보고 가장 결정적인 원인을 찾는다

[원인 분석을 위한 질문과 조건 3가지]

1. LLM한테 출력 기준이나 제약이 없어서, 자유롭게 작성 하는게 원인인가? → **출력 제약을 제공하여 이전과 비교**
2. LLM에게 단순히 비행기 형상 생성에 관한 지식이 부족한 것이 원인인가? → **LLM이 틀리는 특정 부분을 RAG 내부에 그 부분만 청크로 제공 (특정 지식의 예: plan 표현을 잘못함 / 계산식을 잘못 적음 / coverage를 잘못 적음 / LLM이 몰라서 빈칸으로 놔두는 부분)**
+ RAG 한글과 영어 비교
3. LLM에게 특정 부분에 관한 지식을 주고, 출력 제약을 하더라도 잘못 적는가? → **둘 다 제공하면 정확하게 작성하는가?**

1. 출력 제약을 주는 방법 ⇒ LLM이 정해진 규칙에 따라 재작성을 한다.

- LLM에게 해당 내용은 이렇게 적어야 한다~ 라고 알려주면 잘 적지 않을까? 생각함
- 결국 LLM이 추론하는 단계와 기준을 정해주는 방식

출력 내용과 실제 파일 내부

```
def check_rules(value, interpretation):
    """Report rule violations. Values are never patched; the Agent rewrites its own output
    problems = []
    components = value.get('components', [])
    by_id = {c['id']: c for c in interpretation['components']}

    # 3. Registered Geom type identifiers, read from the installed OpenVSP runtime.
    bad = [c.get('openvsp_geom_type') for c in components
           if str(c.get('openvsp_geom_type')) not in REGISTERED]
    if bad:
        problems.append(f'openvsp_geom_type must be one of the Geom type identifiers registered in the OpenVSP runtime. Registered identifiers: {', '.join(REGISTERED)} + '. These are not registered identifiers: ' + ', '.join(repr(b) for b in bad)')

    for component in components:
        cid = component.get('id')
        plan = str(component.get('plan', ''))
        geom_type = str(component.get('openvsp_geom_type'))
        table = FACTS['parms'].get(geom_type, {})

        # 5. The plan must instruct construction, not describe a finished shape.
        if sum(plan.count(verb) for verb in BUILD_VERBS) < 2:
            problems.append(f'{cid} plan must instruct how to build the component in OpenVSP. It must contain at least two verbs: {', '.join(BUILD_VERBS)}')

        # 4. Every requested value must be carried by a Parm that exists on this Geom type
        named = {p for p in ALL_PARMS if re.search(r'(?<![A-Za-z_])' + re.escape(p) + r'(?![A-Za-z_])', plan)}
        wrong = {p for p in named if table and p not in table}
        if not named:
            problems.append(f'{cid} plan does not name any OpenVSP Parm. State which Parm names the requested values, for example the Parm that holds a length')
        if wrong:
            problems.append(f'{cid} plan names Parms that do not exist on {geom_type}: ' + ', '.join(sorted(wrong)))
```

규칙 내용

행	겨냥한 실패	판정 근거	모델에 전달되는 문장
53-56	fuselage, cylinder 등 실재하지 않는 종류명	OpenVSP 등록 25종	"등록된 식별자여야 하며 등록된 철자 그대로 써라. 목록: POD, FUSELAGE, WING.... 등록되지 않은 값: 'cylinder'"
66-68	치수를 서술로만 적고 어느 값이 어디 들어가는지 없음	문장에 Parm 이름이 하나도 없음	"요청된 값은 각각 그 Geom의 Parm이 담는다. 어떤 Parm과 그룹이 각 값을 담는지 밝혀라"
69-71	Diameter, RootChord 같은 지어낸 이름	그 종류의 실제 Parm 표에 없음	"WING에 존재하지 않는 Parm: Height"
72-82	요청에 없던 값을 임의로 끼워 넣음	문장의 모든 숫자가 해석문 값 또는 계산식에 있는지	"요청이 말하지 않았고 계산이 설명하지 않는 숫자: 0.419. 유도들 calculations에 기록하거나 삭제하라"
84-86	부품 ID를 자료 ID 자리에 적음	해석문 부품 ID와 겹치는지	"source_ids는 제공된 자료 식별자여야 하며 부품 ID일 수 없다"
87-90	calculations를 빈 배열로 둠	비었는지	"비어 있다. 요청이 직접 주지 않은 값의 유도를 설명·식·결과·단위로 기록하라"
92-104	coverage 누락, 요구명 변형, 관계 문장 창작	해석문의 부품·요구명과 집합 일치, 관계 문장이 해석문 부분집합인지	"해석문 부품마다 정확히 한 항목이어야 한다", "요구명은 해석문과 정확히 같아야 한다: span, root_chord...", "해석문이 기록한 관계 문장만 쓸 수 있고 지어내거나 바꿔 쓸 수 없다"

1. 출력 제약을 주는 방법 ⇒ LLM이 정해진 규칙에 따라 재작성을 한다.

- LLM에게 해당 내용은 이렇게 적어야 한다~ 라고 알려주면 잘 적지 않을까? 생각함
- 결국 LLM이 추론하는 단계와 기준을 정해주는 방식

출력 내용과 실제 파일 내부

```
def check_rules(value, interpretation):
    """Report rule violations. Values are never patched; the Agent rewrites its own output
    problems = []
    components = value.get('components', [])
    by_id = {c['id']: c for c in interpretation['components']}

    # 3. Registered Geom type identifiers, read from the installed OpenVSP runtime.
    bad = [c.get('openvsp_geom_type') for c in components
           if str(c.get('openvsp_geom_type')) not in REGISTERED]
    if bad:
        problems.append('openvsp_geom_type must be one of the Geom type identifiers registered: ' + ', '.join(REGISTERED) + '. These are not registered identifiers: ' + ', '.join(repr(b) for b in bad))

    for component in components:
        cid = component.get('id')
        plan = str(component.get('plan', ''))
        geom_type = str(component.get('openvsp_geom_type'))
        table = FACTS['parms'].get(geom_type, {})

        # 5. The plan must instruct construction, not describe a finished shape.
        if sum(plan.count(verb) for verb in BUILD_VERBS) < 2:
            problems.append(f'{cid} plan must instruct how to build the component in OpenVSP. '
                            f'(create the Geom, set its Parm, reuse or select sections), '
                            f'not a finished shape. Use construction sentences.')

        # 4. Every requested value must be carried by a Parm that exists on this Geom type
        named = {p for p in ALL_PARMS if re.search(r'(?<![A-Za-z_])' + re.escape(p) + r'(?![A-Za-z_])', plan)}
        wrong = {p for p in named if table and p not in table}
        if not named:
            problems.append(f'{cid} plan does not name any OpenVSP Parm. State which Parm '
                            f'each requested value, for example the Parm that holds a length')
        if wrong:
            problems.append(f'{cid} plan names Parm, but does not exist on {geom_type}: '
                            f'+ ', '.join(sorted(wrong)))
```

해당 내용들

규칙 내용

행	겨냥한 실패	판정 근거	모델에 전달되는 문장
53-56	fuselage, cylinder 등 실재하지 않는 종류명	OpenVSP 등록 25종	"등록된 식별자여야 하며 등록된 철자 그대로 써라. 목록: POD, FUSELAGE, WING.... 등록되지 않은 값: 'cylinder'"
66-68	치수를 서술로만 적고 어느 값이 어디 들어가는지 없음	문장에 Parm 이름이 하나도 없음	"요청된 값은 각각 그 Geom의 Parm이 담는다. 어떤 Parm과 그룹이 각 값을 담는지 밝혀라"
69-71	Diameter, RootChord 같은 지아낸 이름	그 종류의 실제 Parm 표에 없음	"WING에 존재하지 않는 Parm: Height"
72-82	요청에 없던 값을 임의로 끼워 넣음	문장의 모든 숫자가 해석문 값 또는 계산식에 있는지	"요청이 말하지 않았고 계산이 설명하지 않는 숫자: 0.419. 유도들 calculations에 기록하거나 삭제하라"
84-86	부품 ID를 자료 ID 자리에 적음	해석문 부품 ID와 겹치는지	"source_ids는 제공된 자료 식별자여야 하며 부품 ID일 수 없다"
87-90	calculations를 빈 배열로 둠	비었는지	"비어 있다. 요청이 직접 주지 않은 값의 유도를 설명·식·결과·단위로 기록하라"
92-104	coverage 누락, 요구명 변형, 관계 문장 창작	해석문의 부품·요구명과 집합 일치, 관계 문장이 해석문 부분집합인지	"해석문 부품마다 정확히 한 항목이어야 한다", "요구명은 해석문과 정확히 같아야 한다: span, root_chord...", "해석문이 기록한 관계 문장만 쓸 수 있고 지어내거나 바꿔 쓸 수 없다"

1. 출력 제약을 주는 방법 ⇒ LLM이 정해진 규칙에 따라 재작성을 한다.

- 관련 논문 2가지 및 실제 출력 제약 방법 소개

관련 논문

❖ CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing

⇒ 모델이 먼저 출력을 하고, 외부 도구(OpenVSP 부품 종류 이름, Parm등)를 가져와 피드백으로 고쳐쓰는 구조 [뒤에 슬라이드에서..]

❖ SELF-REFINE: Iterative Refinement with Self-Feedback

⇒ 같은 모델이 스스로 피드백(이렇게 적어야 말해준 규칙들)을 만들어 반복 수정하는 구조



[\[2305.11738\] CRITIC: Large Language Models Can Self ...](#)

Z Gou 저술 · 2023 **1002회 인용** — Abstract page for arXiv paper 2305.11738: **CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing.**



[Self-Refine: Iterative Refinement with Self-Feedback](#)

A Madaan 저술 · 2023 **6153회 인용** — The main idea is to **generate an initial output using an LLMs**; then, the same LLMs provides feedback for its output and uses it to refine itself, ...

실제 출력 제약 방법

1. 모델이 일차적으로 형상문 틀만 받은 채로 쪽 쓴다
2. Check_rule이 그 형상문 전체 중에서 OpenVSP 이름 목록과 대조하고, 빠진게 있는지, 규칙에 맞게 썼는지 확인한다.
3. 만약 빠진게 있거나, 잘못 쓴 단어나 문장이 있으면 그걸 정리해서 다시 모델에게 수정 사항을 전달한다
4. 해당 내용(규칙이나 틀린부분들)을 전달 받은 모델은 틀린 부분만 다시 재작성한다.
5. 통과할 때까지 총 6회를 반복한다.

1. 출력 제약을 주는 방법 ⇒ LLM이 정해진 규칙에 따라 재작성을 한다.

- OpenVSP에서 단어를 어떤 것을 들고 오고, 어떻게 사용하는지

OpenVSP를 실제로 실행하는 파일(vsp_Fact)

```
1  """Read Geom types and Parm tables from the installed OpenVSP, never from a fixed list.
2
3  The probe adds one Geom of each requested type in a fresh model and reports the
4  Parm names, groups and limits that OpenVSP itself registers. Nothing here decides
5  which Geom or Parm a request needs; it only reports what exists.
6  """
7  from __future__ import annotations
8
9  import json
10 import os
11 from pathlib import Path
12 import subprocess
13 import tempfile
14
15 ROOT = Path(__file__).resolve().parents[1]
16 PROBE = """
17 void dumpParms(string gid, string tag)
18 {
19     array<string>@ ids = GetGeomParmIDs( gid );
20     for (uint i = 0; i < ids.length(); i++)
21     {
22         Print( tag + "|" + GetParmDisplayGroupName( ids[i] ) + "|" + GetParmName( ids[i] ) + "|" +
23             GetParmLowerLimit( ids[i] ) + "|" + GetParmUpperLimit( ids[i] ) );
24     }
25 }
26
27 void main()
28 {
29     array<string>@ types = GetGeomTypes();
30     for (uint t = 0; t < types.length(); t++) { Print( "TYPE|" + types[t] ); }
31
32     for (uint p = 0; p < types.length(); p++)
33     {
34         VSPRenew();
35         string gid = AddGeom( types[p] );
36         if ( gid.length() == 0 ) { continue; }
37         Update();
38     }
39 }
```

OpenVSP 지식을 가져와 검증?

1. 왼쪽 코드 파일을 통해서 OpenVSP를 직접 실행
2. 검증에 사용하기 위해서 부품 종류 이름, Parm(세부 사항 이름)들을 가져온다
[parm 예: 날개의 span, Root_chord 같은 이름, 소속 그룹, 허용 범위 등]

왜 사용하는가?:

단어 같은 경우를 gpt-oss가 openVSP에 없는 cylinder 같은 단어를 쓰는 경우가 많아서 전체 plan에서 그런 단어를 쓰지 못하도록 검증하고 제거한다.

[아래는 가져온 단어들을 검증하기 위한 명령 코드: check rule]

```
table = FACTS['parms'].get(geom_type, {}) # 그 부품 종류가 실제로 가진 Parm 표
named = {p for p in ALL_PARMS # plan 문장에서 Parm 이름처럼 보이는 것 수집
         if re.search(r'(?![A-Za-z_])' + re.escape(p) + r'(?![A-Za-z_])', plan)}
wrong = {p for p in named if table and p not in table} # 그중 그 종류에 없는 것
```

1. 출력 제약을 주는 방법 ⇒ LLM이 정해진 규칙에 따라 재작성을 한다.

- OpenVSP에서 단어를 어떤 것을 들고 오고, 어떻게 사용하는지

OpenVSP를 실제로 실행하는 파일(vsp_Fact)

```
TYPE|POD
TYPE|FUSELAGE
TYPE|WING
TYPE|STACK
TYPE|BLANK
TYPE|ELLIPSOID
TYPE|BODYOFREVOLUTION
TYPE|HUMAN
TYPE|PROP
TYPE|GEAR
TYPE|HINGE
TYPE|CONFORMAL
TYPE|ROUTING
TYPE|AUXILIARY
TYPE|COBRA
TYPE|OnOffExample
TYPE|PodMan
TYPE|SeatGroup
TYPE|Cone
TYPE|Seat
TYPE|BoxGroup
TYPE|Box
TYPE|TransportFuse
TYPE|Disk
TYPE|Duct
GEOM|FUSELAGE|XForm|X_Location|-1e+12|1e+12
GEOM|FUSELAGE|XForm|Y_Location|-1e+12|1e+12
GEOM|FUSELAGE|XForm|Z_Location|-1e+12|1e+12
GEOM|FUSELAGE|XForm|X_Rotation|-180|180
GEOM|FUSELAGE|XForm|Y_Rotation|-180|180
GEOM|FUSELAGE|XForm|Z_Rotation|-180|180
GEOM|FUSELAGE|XForm|Origin|0|1
GEOM|FUSELAGE|XForm|X_Rel_Location|-1e+12|1e+12
GEOM|FUSELAGE|XForm|Y_Rel_Location|-1e+12|1e+12
GEOM|FUSELAGE|XForm|Z_Rel_Location|-1e+12|1e+12
GEOM|FUSELAGE|XForm|X_Rel_Rotation|-180|180
GEOM|FUSELAGE|XForm|Y_Rel_Rotation|-180|180
GEOM|FUSELAGE|XForm|Z_Rel_Rotation|-180|180
```

OpenVSP 지식을 가져와 검증?

1. 왼쪽 코드 파일을 통해서 OpenVSP를 직접 실행
2. 검증에 사용하기 위해서 부품 종류 이름, Parm(세부 사항 이름)들을 가져온다
[parm 예: 날개의 span, Root_chord 같은 이름, 소속 그룹, 허용 범위 등]

왜 사용하는가?:

단어 같은 경우를 gpt-oss가 openVSP에 없는 cylinder 같은 단어를 쓰는 경우가 많아서 전체 plan에서 그런 단어를 쓰지 못하도록 검증하고 제거한다.

[아래는 가져온 단어들을 검증하기 위한 명령 코드: check rule]

```
table = FACTS['parms'].get(geom_type, {}) # 그 부품 종류가 실제로 가진 Parm 표
named = {p for p in ALL_PARMS # plan 문장에서 Parm 이름처럼 보이는 것 수집
          if re.search(r'(?![A-Za-z_])' + re.escape(p) + r'(?![A-Za-z_])', plan)}
wrong = {p for p in named if table and p not in table} # 그중 그 종류에 없는 것
```

1. 출력 제약을 주는 방법 ⇒ LLM이 정해진 규칙에 따라 재작성을 한다.

- 최종 결과와 아직 해결되지 못한 점

형상 틀만 vs 규칙 검사

❖ 정답지 기준을 45개의 파트로 나눠서 비교

#	항목	형상틀만 (21/45)	규칙 검사 후 재작성 (35/45)	정답지
1	부품 구성	4부품 ✓	4부품 ✓	4부품
2	전체 치수	전부 일치 ✓	전부 일치 ✓	동일
3	Geom 식별자	"fuselage" X	"FUSELAGE" ✓ 3/3	"FUSELAGE"
4	Parm 지정	없음 X	실제 Parm 이름 기재 ✓	Length · Span · Root_Chord 등
5	문장 성격	완성 형상 묘사 X	구현 지시문으로 전환 ✓	"Create... Set..."
6	자율 설계 추가	요청에 없는 서술 X	설명 없는 숫자 제거 ✓	근거 있는 선택만
7	배치·대칭	Z값 없음, 대칭 미언급 X	단일 핀 대칭 3/3, 높이 기준 2/3 △	Z_Rel_Location 0, Sym_Planar_Flag 0
8	source_ids	[] X	채움 ✓ 3/3	자료 ID
9	calculations	[] X	채움 ✓ 3/3	5 m·2 m
10	coverage	[] X	채움 ✓ 3/3	1부품 요구 관계
11	후퇴각 기준	없음 X	없음 X 0/3	Sweep_Location 0.25
12	익형 Parm	없음 X	없음 X 0/3	Camber 0 · ThickChord 0.12
13	단면·드라이버	없음 X	없음 X 0/3	XSec_1 ·드라이버 3개

“아직 실패한 부분”

아직 안되는건 왜?

- 개념을 몰라서 또는 빈칸으로 둠 (1·2·3·4·10·11·12):
편측 스패, 대칭 해제·회전, 배치 Parm, 실행 지시 어투, 계산 기준, 관계 기재.
- 자세한 명칭들을 몰라 자기 말로 씀 (5·6·7·8·9 그룹명):
openVSP에서 못 가져온 부분들 따로 OpenVSP 사전에서 발췌하여 칭크로 넣어줄 예정

#	항목	정답지	이전 결과	차이	채점 반영
1	주익 스패	Span 5 m (편측) → 전체 10 m	"total span of 10 m"에 Span 이름 부여	코드로 옮기면 20 m	X 미반영(5번 3점)
2	수평포리 스패	Span 2 m → 전체 4 m	"total span of 4 m"	코드로 옮기면 8 m	X 미반영(6번 3점)
3	수직포리 단일 핀	Sym_Planar_Flag 0 + X_Rel_Rotation +90 deg	"extends upward along +Z" (서술만)	지정 수단 없음	X 미반영(8번 3점)
4	배치	X_Rel_Location 4, Y_Rel_Location 0, Z_Rel_Location 0 in XForm, zero rotation	"root leading edge is located at (4,0,0)"	어느 Parm인지 없음	X 채점 항목 없음
5	후퇴각	Sweep 15 deg + Sweep_Location 0.25	Sweep25Pct (없는 이름)	이름 지어냄	✓ 10번 0점
6	익형	Camber 0, ThickChord 0.12 on XSecCurve_0/1	AirfoilRoot, AirfoilTip (없는 이름)	이름 지어냄	✓ 11번 0점
7	단면·드라이버	XSec_1 재사용, 드라이버 3개 선택	WingParams (없는 그룹), 드라이버 언급 없음	누락 + 지어냄	✓ 12번 0점
8	동체 단면	기분 5단면 유지, XSecCurve_1/2/3 예, Ellipse_Width · Ellipse_Height 1.5	MaxWidth, MaxHeight (없는 이름)	이름 지어냄	X 채점 항목 없음
9	동체 길이	Length in Design 10 m	Length (그룹은 FuselageParams에 포함)	이름만 맞춤	✓ 4번 3점

10	문장 성격	Create / Set / Reuse / Retain 실행 지시	"...는 ...이다" 완성형상 묘사	무엇을 건드리지 말지 없음	X 채점 항목 없음
11	calculations	실제 유도(편측 5 m 등)	$(Span/2) \cdot \tan(Sweep25Pct) = 1.357$	Span에 전체값을 넣어 기준 어긋남, 값도 $5 \cdot \tan 15^\circ = 1.34$ 와 불일치	X 미반영(13번 3점)
12	coverage 관계	해석문 관계 문장 기재	relationship_texts: [] 전부 빈 배열	관계 미기재	X 미반영(14번 3점)
13	부품 구성 종류	4부품, FUSELAGE / WING ×3	동일	—	✓ 2·3번 3점
14	치수 숫자	10, 1.5, 2, 1, 15, 1.2, 0.6, 1.8, 0.7, 25	전부 등장	—	✓ 4~7번 3점

■ 다음주 계획 보고

1. 형상 계획문 작성할 때 사용한 기법들을 통해서, 동시에 코드 작성 단계도 적용 및 확인 중
2. 마지막에 아직 성공하지 못한 부분은 RAG에 청크를 넣어서 해결을 하는지 확인 중
3. 형상 단계 통과되면, 코드 작성 단계도 빠르게 해결
4. 모두 통과 시, 전체 파이프라인 검사 후
 - a. 자세한 파라미터 수치도 적절히 잘 교체하여 반영하는지 확인
 - b. 다른 부품들도 추가가 가능한지
 - c. 다른 비행기 형상 부품들도 형상 생성이 가능한지 테스트

Any Questions?

이름 최준석
소속 한양대학교 기계공학과
이메일 chw10109@hanyang.ac.kr



*Intelligent Design
for Engineering
Applications Lab*

